

FYP REPORT

by

Yong Sheng Kai

1201200414

Session 2025/2026

The project report is prepared for
Faculty of Artificial Intelligence and Engineering
Multimedia University
in partial fulfilment for
Bachelor of Science (Honours) in Intelligent Robotics

FACULTY OF ARTIFICIAL INTELLIGENCE AND ENGINEERING
MULTIMEDIA UNIVERSITY

June 2026

© 2026 Universiti Telekom Sdn. Bhd. ALL RIGHTS RESERVED.

Copyright of this report belongs to Universiti Telekom Sdn. Bhd. as qualified by Regulation 7.2 (c) of the Multimedia University Intellectual Property and Commercialisation Policy. No part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), or for any purpose, without the express written permission of Universiti Telekom Sdn. Bhd. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

DECLARATION

I hereby declare that this work has been done by myself and no portion of the work contained in this report has been submitted in support of any application for any other degree or qualification of this or any other university or institute of learning.

I also declare that pursuant to the provisions of the Copyright Act 1987, I have not engaged in any unauthorised act of copying or reproducing or attempt to copy / reproduce or cause to copy / reproduce or permit the copying / reproducing or the sharing and / or downloading of any copyrighted material or an attempt to do so whether by use of the University's facilities or outside networks / facilities whether in hard copy or soft copy format, of any material protected under the provisions of sections 3 and 7 of the Act whether for payment or otherwise save as specifically provided for therein. This shall include but not be limited to any lecture notes, course packs, thesis, text books, exam questions, any works of authorship fixed in any tangible medium of expression whether provided by the University or otherwise.

I hereby further declare that in the event of any infringement of the provisions of the Act whether knowingly or unknowingly the University shall not be liable for the same in any manner whatsoever and undertakes to indemnify and keep indemnified the University against all such claims and actions.

Signature: _____

Name: Yong Sheng Kai

Student ID: 1201200414

Date:

ACKNOWLEDGEMENT

ABSTRACT

TABLE OF CONTENTS

CHAPTER 1	INTRODUCTION.....	1
1.1	Overview-not.....	1
1.2	Problem Statements -not	1
1.3	Objectives-not	2
1.4	Project Scope-not	2
CHAPTER 2	LITERATURE REVIEW-not.....	3
2.1	Guides on Writing Reviews of Literature-not.....	4
2.1.1	Introduction Section-not.....	4
2.1.2	Literature Review Section-not	4
2.2	References and Citations-not	4
CHAPTER 3	DETAILS OF THE DESIGN.....	5
3.1	Hardware Design-not	5
3.2	Mechanical Design-not	8
3.2.1	2804 Gimbal Brushless Direct Current (BLDC) Motor.....	9
3.2.2	MT6701 Magnetic Encoder 1024 Pulse Per Rotation (PPR).....	11
3.2.3	3s 11.1V 550mah Lithium Polymer (LiPo) battery.....	12
3.2.4	MG996R Servo Motor	13
3.2.5	SimpleFoc Mini V1.0 Motor Driver	14
3.2.6	MP2236 Synchronous Step-Down Converter.....	15
3.2.7	MPU6050 6-axis motion tracking sensor (MPU6050)	16

3.2.8	ESP32-S3 Dev-C board.....	17
3.2.9	3D-Printed Polylactic Acid (PLA)body	18
3.3	Power distribution and electronic design	20
3.3.1	Electrical System	20
3.3.2	Overall Electronic Circuit Diagram	21
CHAPTER 4 ROBOT MATHEMATIC MODEL DESIGN		24
4.1	Linear Equations of Motion for TWSB Robot.....	24
4.2	Variable geometry mathematic model build	37
4.3	Free RTOS.....	39
4.4	Kalman Pitch estimation-not.....	39
4.5	LQR Implementation-not	40
4.6	Four profile gain switches-not.....	40
4.7	Yaw loop-not.....	40
4.8	Servo library	41
CHAPTER 5 DATA PRESENTATION AND DISCUSSION OF FINDINGS...		44
CHAPTER 6 CONCLUSIONS.....		51
6.1	Summary and Conclusions-not	51
6.2	Areas of Future Research -not.....	51

LIST OF FIGURES-not

Figure 3.1: Example of figure and its caption.....	Error! Bookmark not defined.
Figure 3.2: Example of acceptable quality image.	Error! Bookmark not defined.
Figure 3.3: Example of bad quality image (enlarged beyond their print resolution)	Error! Bookmark not defined.

LIST OF TABLES-not

Table 3.1: Table Styles **Error! Bookmark not defined.**

Table 4.1: General Considerations when Using Table and Graphs**Error! Bookmark not defined.**

LIST OF ABBREVIATIONS-not

FYP	Final Year Project
ADC	Analog to Digital Converter
DAC	Digital to Analog Converter
DSP	Digital Signal Processing
PLA	Polylactic Acid
LDO	Low-Dropout Regulator
LiPo	Lithium Polymer
MCU	
I2C	
BLDC	Brushless Direct Current
PPR	Pulse Per Rotation
AWG	American Wire Gauge

CHAPTER 1 INTRODUCTION

1.1 Overview-not

Mobile robotics typically faces a trade-off between operational speed and terrain mobility. While traditional wheeled robots exhibit high efficiency on flat surfaces, they often struggle to navigate rough terrain. Conversely, legged robots are adept at handling obstacles but lack the speed and energy efficiency of wheeled systems. This project introduces a hybrid solution: a two-wheel-legged robot. This design utilizes a four-bar linkage mechanism to actively alter the robot's height and Center of Gravity (CoG). This capability allows the robot to squat to navigate under height-constrained obstacles and adjust its leg geometry to traverse uneven terrain. Crucially, these maneuvers are performed while maintaining a self-balancing state, employing the Field Oriented Control (FOC) algorithm to ensure smooth, high-torque motor control at low speeds.

1.2 Problem Statements -not

- **Fixed Geometry Limitations:** Traditional self-balancing robots cannot navigate height-constrained environments (e.g., under tables or pipes).
- **Motor Control Instability:** Standard BLDC motor control often suffers from "cogging" or jitter at low speeds, which destabilizes self-balancing robots.
- **Computational Load:** Handling complex balancing algorithms and kinematics simultaneously on a single microcontroller often leads to loop-time latency and instability. This project addresses these issues by implementing a distributed control architecture (ESP32 + Arduino Nanos) and a mechanical crouching mechanism.

1.3 Objectives-not

- To design a variable geometry chassis by using four-bar linkage mechanism to allow height adjustment while maintaining structural strength.
- To design a distributed electronic architecture by using ESP32 to handle the high level computing where Arduino nano executes low-level motor actuation.
- To implement SimpleFOC algorithm and get a smooth high-torque motor to achieve self-balancing at low speed.

1.4 Project Scope-not

CHAPTER 2 LITERATURE REVIEW-NOT

Field Oriented Control (FOC) algorithm for low-speed control Conventional control methods for Brushless DC (BLDC) motors, such as six-step commutation, often exhibit significant torque ripple and mechanical jitter at low rotational speeds. These characteristics are detrimental to the performance of self-balancing robots, where precise control is required. To address these limitations, this project implements the SimpleFOC algorithm. As detailed in the SimpleFOC documentation, this approach utilizes Field Oriented Control (FOC) to precisely align the motor magnetic field with the rotor position. This alignment facilitates smooth operation and high torque generation even at near-zero velocities, a critical requirement for maintaining the robot's equilibrium without inducing mechanical oscillation.

Distributed Control via UART Communication To distribute the computational load of balancing and kinematics, this project employs a distributed architecture that separates high-level processing from low level actuation. The system utilizes an ESP32 as the high-level computing unit, which sends commands to the low-level actuation microcontrollers, specifically Arduino Nanos. The Nanos receive these instructions and drive the motors and servos accordingly. The UART serial protocol is used to facilitate communication between the master and slave units. Additionally, a logic level converter is integrated to bridge the voltage difference between the 3.3V ESP32 and the 5V Arduino Nano, ensuring reliable communication without damaging the components.

Sensor Fusion and control strategies To balance a 2 wheeled self-balancing robot required precise control and attitude estimation. To get the angle of the robot, MPU6050 Inertia Measurement Unit will be used in this project. It combined 3 axis accelerometers and a 3-axis gyroscope. But research by Verma & Rajnakar (2024) highlights that using the sensor individually will cause some issue over time, due to the accelerometers are sensitive to vibration, while the gyroscopes will “drift” over time. To solve this problem, data from both sensors must be combined and using digital filter like complementary filter or Kalman filtering to provide smoother and more accurate angle data for the robot. PID algorithms will also be required in this project. It constantly calculates the error between the target angle and the current angles of the robot and adjusts the motor output to minimize deviation and maintain balance.

This chapter demonstrates your ability to review, and to report on relevant literature and past work on your given topic. A well written literature review:

- allows the reader to easily place the focus of your FYP project within the context of the wider academic community in your field;
- reports your critical review of the relevant literature; and
- Identifies a gap within that literature that your research will attempt to address.

2.1 Guides on Writing Reviews of Literature-not

2.1.1 Introduction Section-not

2.1.2 Literature Review Section-not

2.2 References and Citations-not

CHAPTER 3 DETAILS OF THE DESIGN

3.1 Hardware Design-not

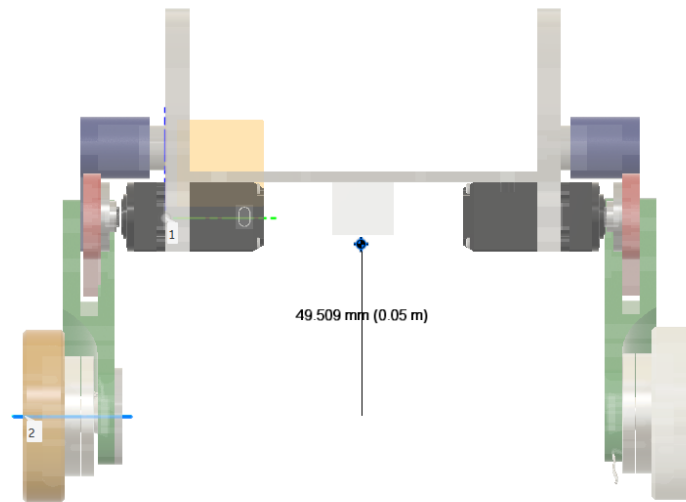


The figure shows the cad design of the robot body in Fusion 360.

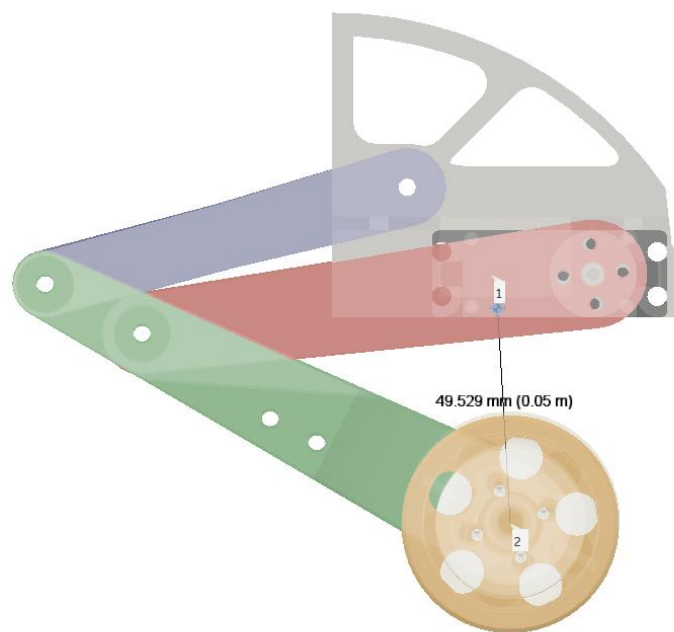
This chapter covers the components used to build the robot, including the mechanical design, electronic power rails, and wiring diagram. In this chapter will also discuss about why the components are selected of this project. How to connect from Microcontroller to sensor and how to power the sensors.

The robot body has run to simulation and cad design to ensure the robot body were

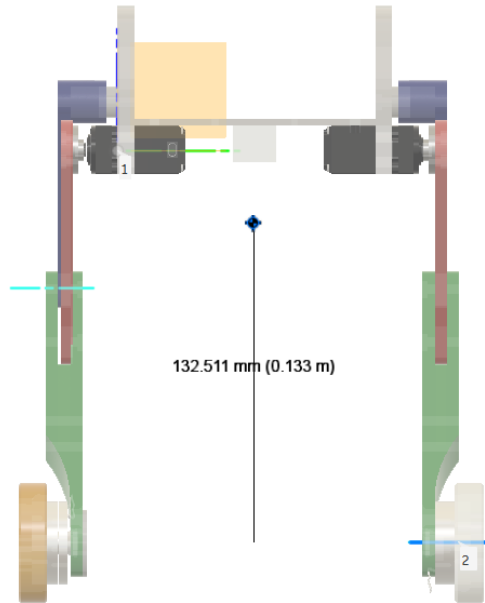
Those simulations will be show in section below.



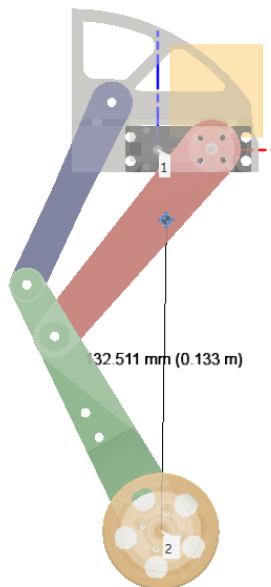
The figure above shows the distance between Cog and the wheel axis of the robot when squashed down at the front view. The Cog is generated in the Fusion360 3D Modelling Software. The side view will show at the figure below



In this figure, the angle between Cog and the wheel axis are slightly tilt a bit backward, this is due to the linkage of the robot are mostly at the rear of the robot. The Cog will move as the linkage move towards to the Coc.



The figure above shows the distance between Cog and the wheel axis of the robot when stand up at the front view. The Cog is generated in the Fusion360 3D Modelling Software.

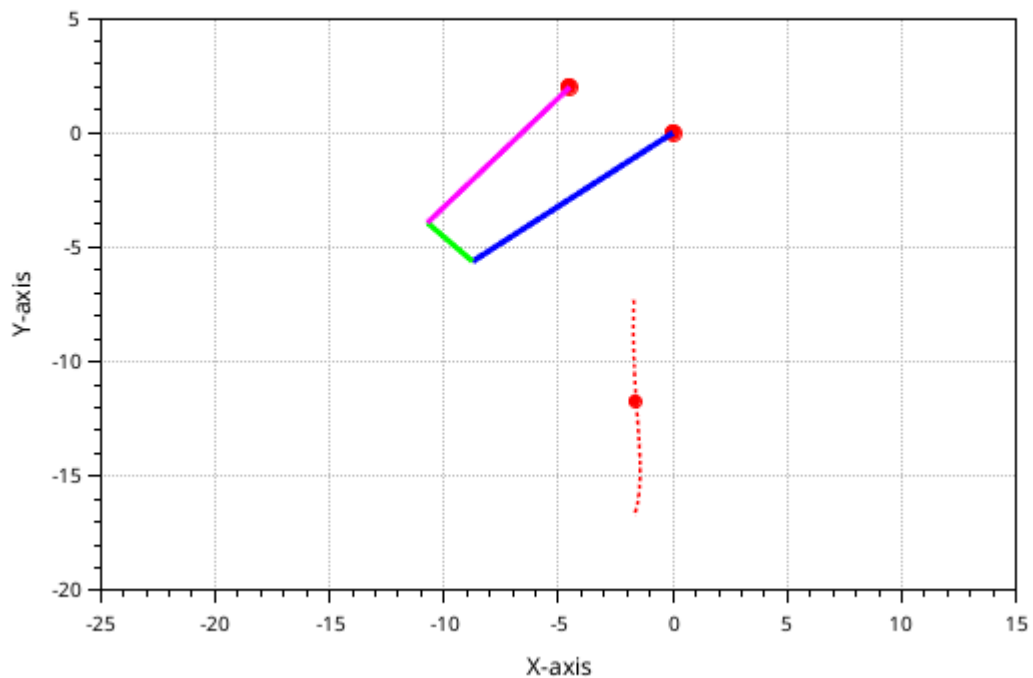


In the figure above show the distance between Cog and the wheel axis, The Cog being move in front compared to the squad down state. This is due to the linkage are move from rear of the robot to below of the robot.

3.2 Mechanical Design-not

4 Bar Linkage Design

4-Bar Linkage Simulation



The figure above shows a motion simulation of the variable-geometry part of the robot in Scilab. It converts the rotational motion of the MG996R Servo Motor into vertical motion. From the figure above, the red dot connected to the pink line is the pivot joint, which is a free joint.

The pink line represents the follower bar; the blue line represents the driver bar, and the green line represents the coupler; the wheel position of the robot is the red dot on

the dotted red line. The dotted red line represents where the end of the arm can reach.

The red dot connected to the blue line represents the rotating shaft of the MG996R Servo Motor. Both red dots are fixed to the body, which can also be referred to as the ground.

3.2.1 2804 Gimbal Brushless Direct Current (BLDC) Motor

In this project, a small, high-torque, and precise motor was needed. For a Direct Current (DC) motor to produce the same torque as a BLDC Motor of the same size, a reducer would be required. However, a reducer introduces backlash due to its gears and creates a dead zone during acceleration and deceleration, which causes minor errors that the robot must correct by rocking back and forth.



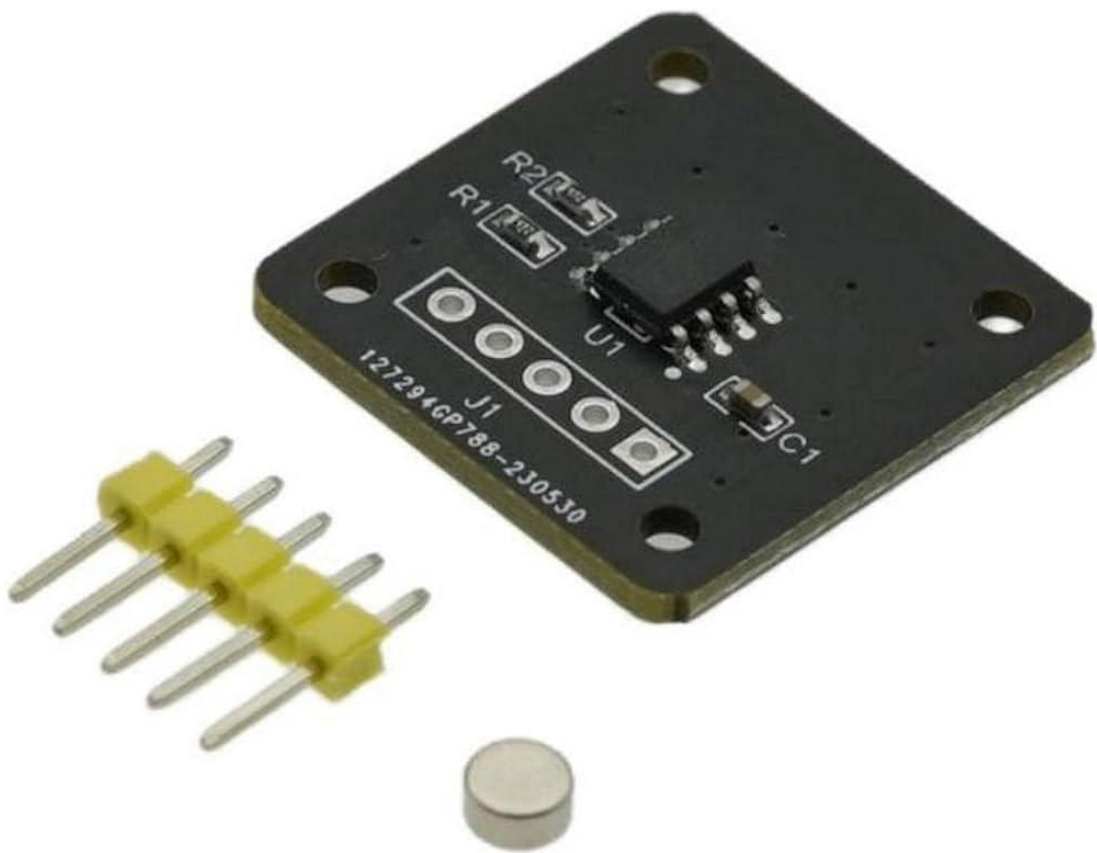
The figure shows the BLDC Motor chosen for this project; it is a gimbal motor, which has a lower KV rating and higher torque, as required for this application. The specifications are shown below. Note that the datasheet states a phase resistance of 2.55Ω , while the value measured in this project was 2.9Ω .

Rated Voltage	7.4-16	V
KV Rating	220	KV
Phase Resistance	2.9Ω	Ω
Winding Resistance	5.1	Ω
Torque	0.3	KG.CM
Number of slots	12	-
Number of poles	14	-
Weight	37	G
Rated Current	0.5	A
Max Current	2	A

<https://www.dfrobot.com/product-3005.html?srltid=AfmBOoqh9SV1UPO6eH13G7AzSGtvDVaWxprmdXcDMPQL>

3.2.2 MT6701 Magnetic Encoder 1024 Pulse Per Rotation (PPR)

To perform Field-oriented Control (FOC), an encoder is needed to let the Microcontroller (MCU) know the exact position of the rotor. With the position of the rotor, the MCU can calculate the magnetic field angle, optimise torque generation, and maximise the motor efficiency.



The figure shows the Magnetic Encoder, which is used in this project. It is MT6701, produced by MagnTek. It has a native 14-bit resolution, corresponding to 0.022° per step. In this project, its ABZ incremental output is configured at 1024 PPR, which after $4\times$ quadrature decoding gives 4096 counts per revolution, an effective resolution of 0.088° . The specifications of the MT6701 Encoder are shown below

https://uploadcdn.oneyac.com/attachments/files/brand_pdf/magntek/F3/CA/MT6701QT-STD.pdf

3.2.3 3s 11.1V 550mah Lithium Polymer (LiPo) battery



The figure above shows the battery used for this project. This battery was selected around the 2804 gimbal BLDC motor, which has a rated current of 0.5A and a maximum current of 2A. To maximise the power of the motor, increasing the supply voltage is the most effective approach. The motor can run on 7.4V to 16V, which means a 2s to 4s LiPo battery can be used in the project. A 3s LiPo battery was chosen instead of a 4s LiPo battery because 4s batteries are harder to obtain than 3s batteries. The capacity of 550mAh was chosen because this battery offers a better weight-to-runtime ratio for the robot.

Switching from a 3s 3000mAh LiPo battery to a 3s 550mAh LiPo battery cut the weight of the robot from 630g to 504g, a 20% reduction in body mass, allowing the 2804 gimbal motor to use less torque to balance and drive the robot. The 550mAh battery is still able to run the robot for around 10 minutes due to the efficiency of the BLDC Motor.

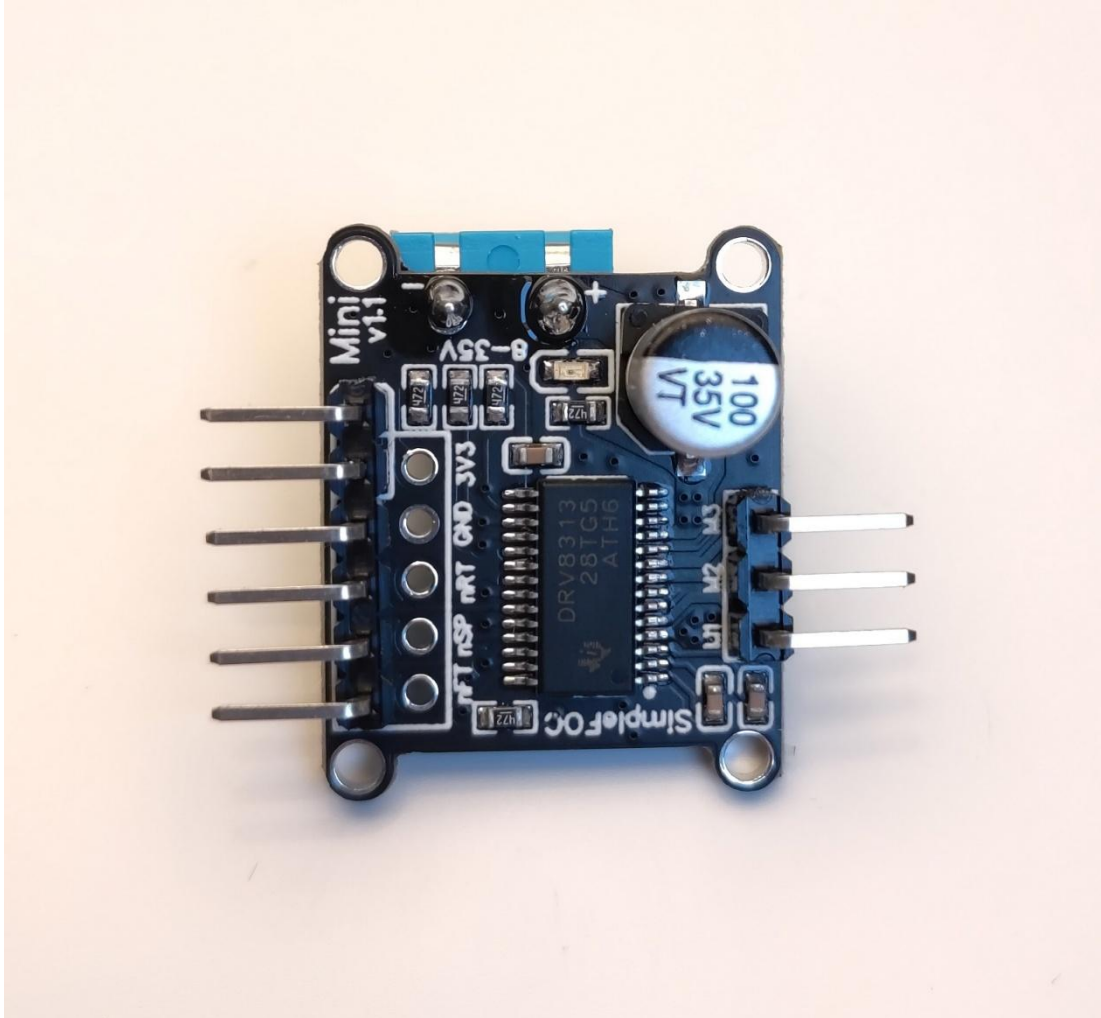
3.2.4 MG996R Servo Motor



The figure above shows the Servo Motor used in this project, the MG996R. This Servo Motor was chosen because it offers a good balance between price and torque. It can

provide 9.8kg.cm at 4.8V, which is sufficient for it to act as a hip motor that moves the robot body up and down.

3.2.5 SimpleFoc Mini V1.0 Motor Driver



SimpleFoc Mini V1 Motor Driver is an electronic speed controller (ESC) for BLDC Motors. It controls the BLDC Motor movement by energising each phase with accurate timing. This specific ESC was chosen for this project due to its small module size of only 26mm by 21mm, which offers more mounting flexibility. Despite its small size, the board is based on the TI DRV8313 chip, which is rated for a maximum current of 3A in its datasheet, while the SimpleFOC documentation rates the board more

conservatively at 2.5A. Either rating is more than sufficient for the 2804 Gimbal BLDC Motor mentioned above.

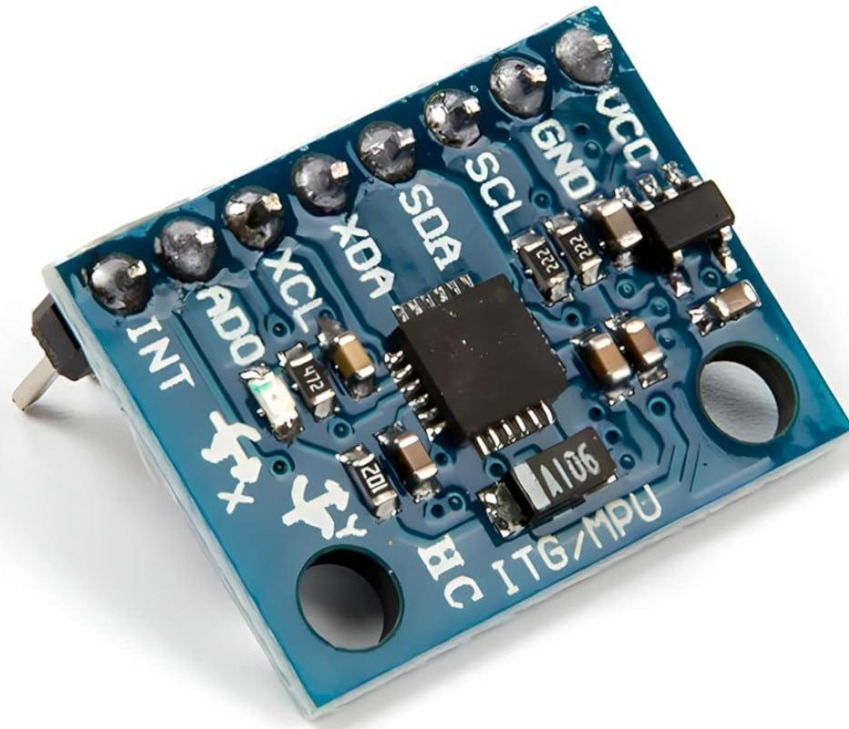
3.2.6 MP2236 Synchronous Step-Down Converter

As the above MG996R Servo Motor section mentions, the rated voltage of the MG996R Servo Motor is 4.8V to 7.2V. To step down the voltage from the battery to the rated voltage required by the MG996R Servo Motor, a voltage regulator is needed. Since the MG996R Servo Motor can draw 2.5A when stalled, a high-power voltage regulator was required for this project.



The figure above shows the voltage regulator module, an MP2236 module pre-made by Fighting embedded development studio. It has 6A max current capability, which is more than sufficient for 2 MG996R Servo Motors. As the data sheet mentions, MP2236 can step down the voltage from a minimum of 3V to a maximum of 18V. Efficiency is around 88% to 95% when stepping down from 12V to 5V.

3.2.7 MPU6050 6-axis motion tracking sensor (MPU6050)



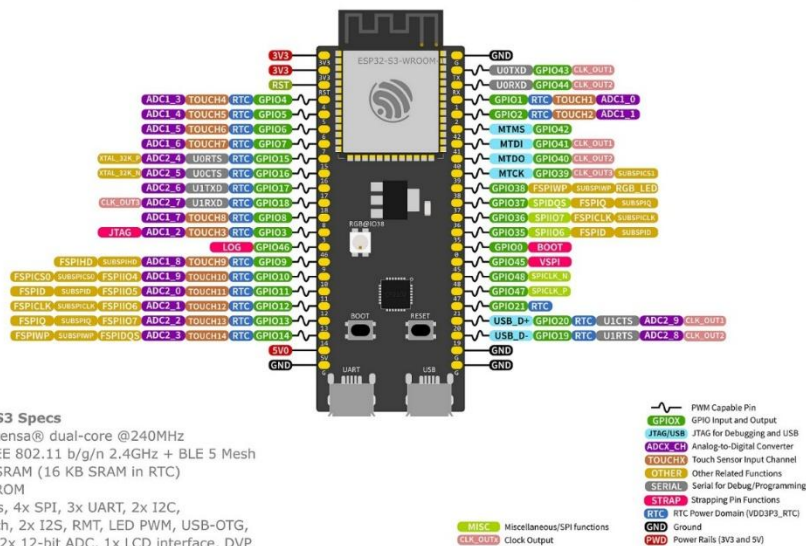
The figure above shows the angle sensor used in this project, which is the MPU6050. It is a common 6-axis motion tracking sensor. It is easy to obtain, and there is plenty of community support, which makes it very suitable for this project. It provides a 3-axis gyroscope and a 3-axis accelerometer to measure the angular velocity and acceleration around the X, Y, and Z axes. It uses the Inter-Integrated Circuit (I2C) protocol to communicate with the ESP32-S3, using only 2 pins.

<https://cdn.sparkfun.com/datasheets/Sensors/Accelerometers/RM-MPU-6000A.pdf>

3.2.8 ESP32-S3 Dev-C board

ESP32-S3 is a module produced by Espressif Systems. It offers 2 processor cores and operates at up to 240MHz, which is much faster than the Atmega328p. With this processing power, the robot can run the Kalman filter, complementary filter, and SimpleFoc algorithm flawlessly. It also offers 2.4GHz Wi-Fi and Bluetooth 5 Low Energy (BLE) for extra connectivity. The board used in this project is the ESP32-S3 Dev-C kit, which is a modified version of the original ESP32-S3 Dev-C board. It uses the original module from Espressif Systems for stability and replaces the Micro Universal Serial Bus (USB) port with a USB Type-C port, which makes the connection much more convenient.

ESP32-S3-DevKitC-1



General Purpose Input Output (GPIO) Pin	Pin Assignment
1, 2	MPU6050 SDA and SCL
3, 46	Left Wheel Encoder A and B
17, 18	Right Wheel Encoder A and B
11,12, 13, 14	Left Motor PWM and Enable Pin
16, 15, 7, 6	Right Motor PWM and Enable Pin
4	Left Hip Servo
5	Right Hip Servo

3.2.9 3D-Printed Polylactic Acid (PLA)body

PLA is one of the materials that can be 3D printed on almost all 3D printers. It is good for beginners because PLA provides a good amount of strength, and it won't release

harmful gases like Acrylonitrile Butadiene Styrene (ABS). Another benefit of the PLA used in this project is that it offers many variants to choose from. Beyond different surface finishes and colours, some variants of PLA add chopped Carbon Fiber or Glass Fiber to provide more strength.

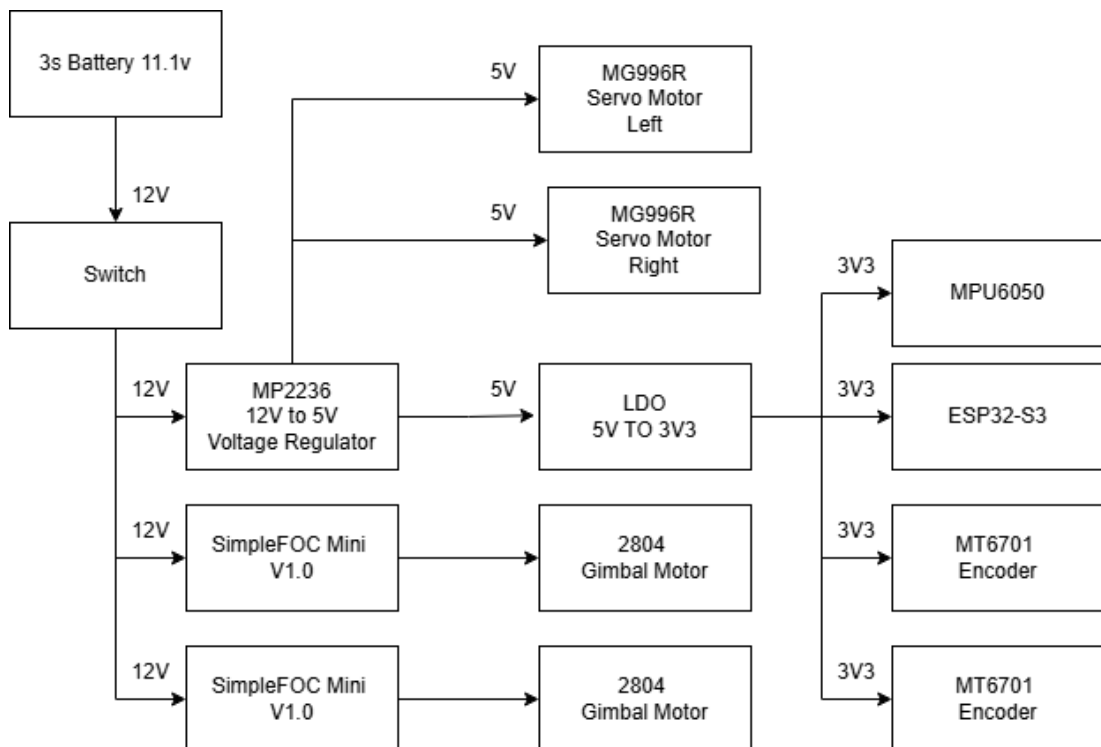
One of the benefits 3D printing offers is rapid prototyping. This reduces the time to fabricate the parts compared to building the robot from an aluminium bar or cutting from Polyvinyl chloride (PVC) board. The robot body can be easily reprinted if any changes are made. 3D printing also eliminates human error during fabrication, as it prints the exact parts according to their models.

3.3 Power distribution and electronic design

This section presents the power rail design and the reasoning behind the electronic component selection.

3.3.1 Electrical System

This section contains Electrical System of the robot and the of the robot.



The figure above shows the power distribution of the robot. The system can be split into 3 power rails, which are listed below:

1. 12v Rail

- Power SimpleFOC Mini v1.0 Driver to drive 2804 Gimbal Motor
- Power MP2236 Voltage Converter to step down the voltage to 5V to power other modules

2. 5V Rail

- Power 2 MG996R Servo Motor

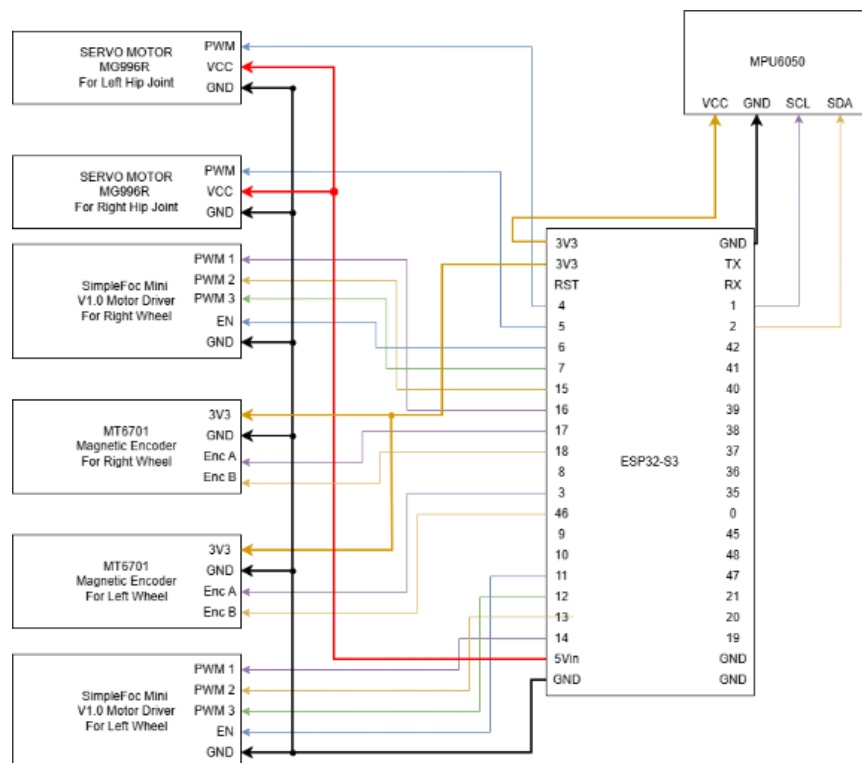
- Power LDO to step down the voltage to 3.3V to power other modules

3. 3V3 Rail

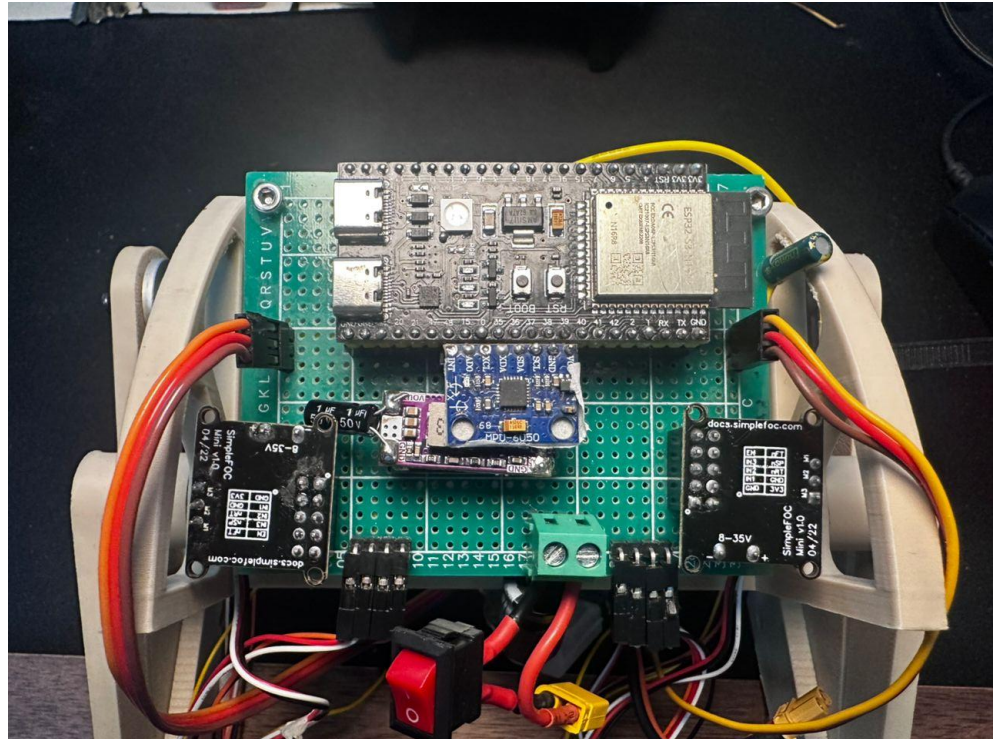
- Power ESP32-S3 Micro Controller (MCU)
- Power MPU6050
- Power MT6701 Magnetic Encoder

3.3V is provided by an LDO called the AMS1117, which is the voltage regulator built into the ESP32-S3 Dev-C board. The LDO receives 5V from the MP2236. The LDO is not powered directly from 12V because an LDO dissipates the excess voltage as heat and stepping from 12V down to 3.3V would generate a lot of heat due to the large voltage difference.

3.3.2 Overall Electronic Circuit Diagram



The figure above shows the overall electronic circuit diagram from the ESP32-S3 to the MG996R Servo motor, SimpleFOC mini V1.0 driver, etc. Point-to-point wiring would be messy and hard to debug. Hence, a prototype board was used to make the connections cleaner and easier to work on.



The figure above shows the Prototype board with all the components on it. The ESP32-S3 Dev-C kit is mounted on the middle top of the prototype board. At the centre of the prototype board, an MPU6050 is mounted to get the 0 offset angle data. There is also an MPU6050 5V Voltage regulator mounted under the MPU6050 to save some space. The 3-pin connectors on the left and right of the board will be the connectors for the Left and Right Hip Servo Motor. The SimpleFOC Mini V1.0 Motor Driver is mounted inverted on the left and right bottom of the prototype board because its pin headers face upward. The MT6701 magnetic encoder for the motor will be the 4-pin connector mounted beside the SimpleFOC Mini V1.0 Motor Driver. Lastly, the green screw terminal is connected to an XT30 connector with a main switch on the negative side. The battery will connect to the XT30 connector to power the whole robot. All the

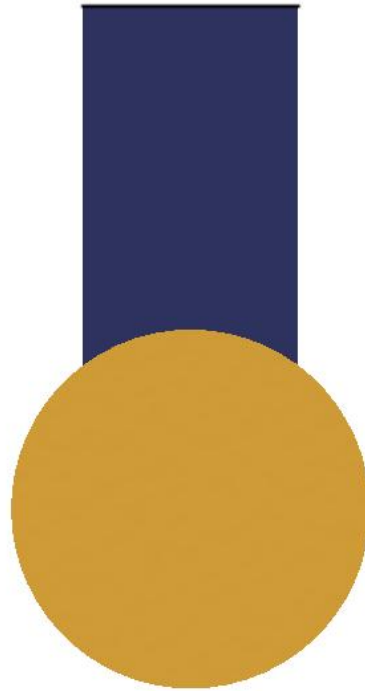
power rails have a capacitor, which acts as an energy reservoir and filters noise on the power rail.

2804 gimbal motor calculation

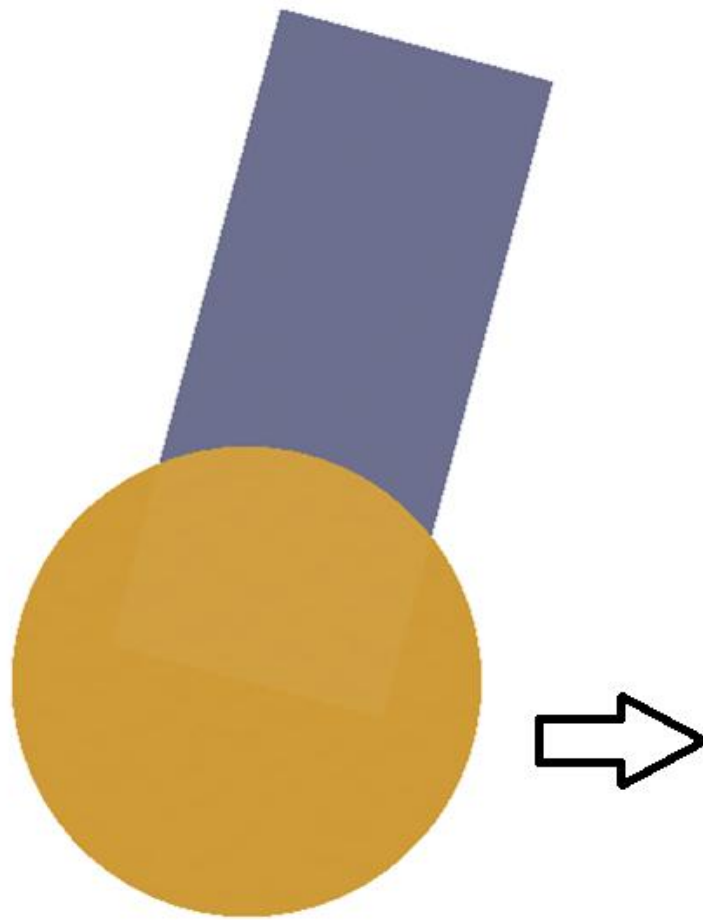
Ldr design to calculate the state spacer vector

CHAPTER 4 ROBOT MATHEMATIC MODEL DESIGN

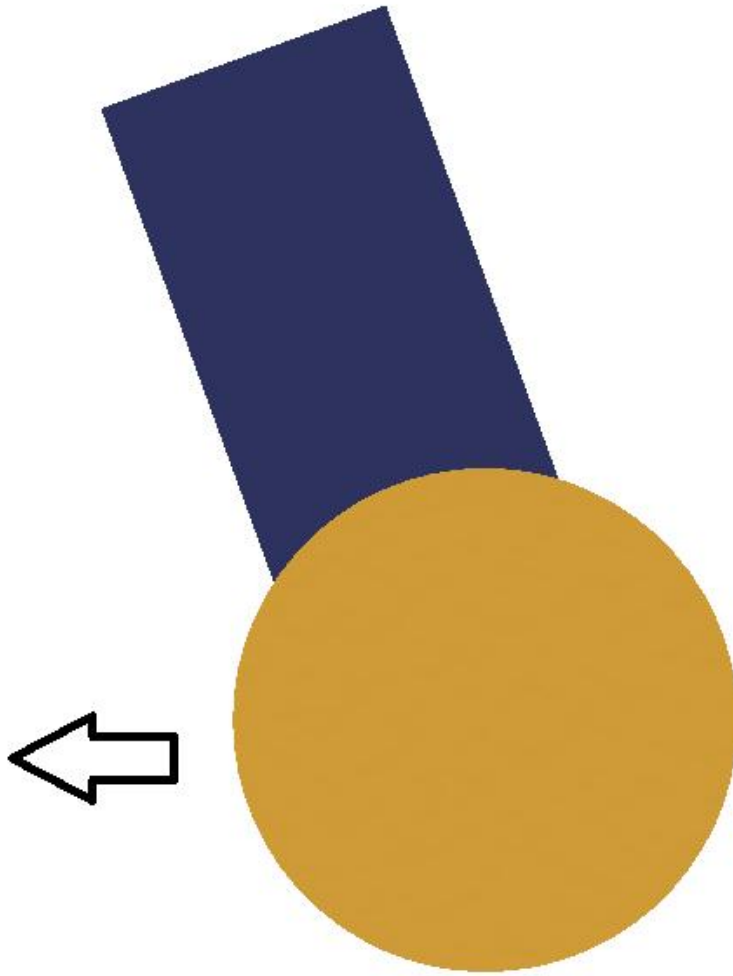
4.1 Linear Equations of Motion for TWSB Robot



The figure above shows the simplify model of the inverse pendulum model. The inverse pendulum model only includes the wheel which is marked as yellow and the rigid body which is marked as blue.

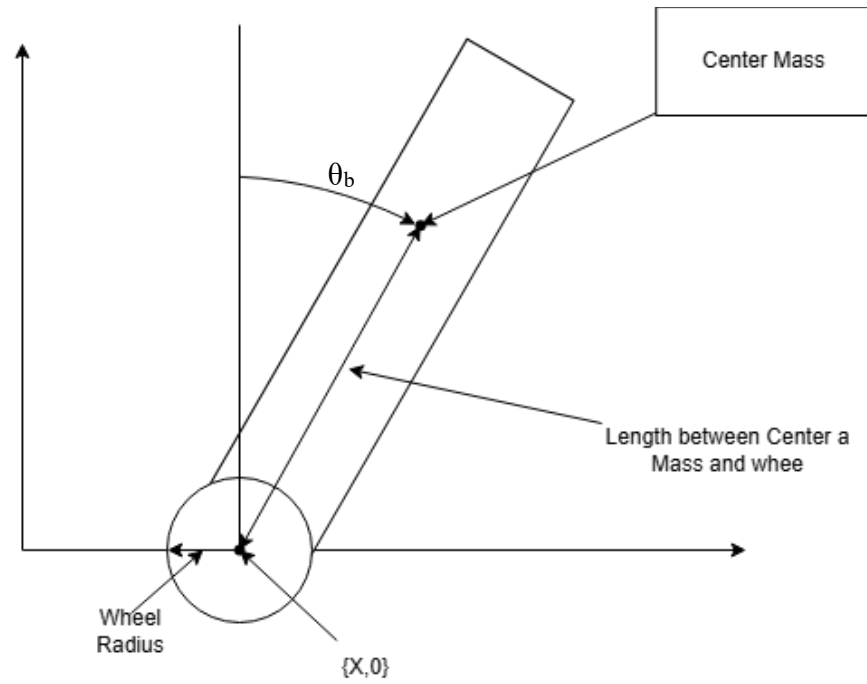


The figure above shows when the body of the robot falls to front side, the wheel attaches to a motor will run forward to catch the fall. And the angle of the robot fall will be call as θ_b .



The figure above shows when the body of the robot falls to back side, the wheel attached to a motor will run backward to catch the fall.

This will be simplest explanation of the inverse pendulum robot.



The figure above shows the parameter used through the report.

R : Wheel radius using symbol R to represent.

l_b : Length between Center mass and wheel axis.

M_b : Mass of the body excludes the wheel.

J_b : Moment of inertia of the body

M_w : Mass of the wheel.

J_w : Moment of inertia of the wheel

θ_b : will be the tilt angle.

And the position of the body center mass can be calculated by, as the center mass will move respect to the wheel center as the body tilt by θ .

$$x_b = x + l_b \sin \theta$$

$$y = l_b \times \cos \theta$$

The velocity of the center mass can be calculated by differentiating both equations with respect to time. Those equations will be shown below.

$$\dot{x}_b = \dot{x} + l_b \times \dot{\theta} \times \cos \theta$$

$$\dot{y} = -l_b \times \dot{\theta} \times \sin \theta$$

Those velocities will be used in the kinetic energy equation below.

Total kinetic energy of the robot is built up by 4 parts, which are wheel translation, wheel rotation, body translation, and body rotation. That part will show as below.

Wheel translation

Wheel rotation

The wheel rotation here is calculated considering only non-wheel slip conditions.

Body translation

Body rotation

J_w can be calculated as a simple cylinder model. As the equation below

$$J_w = \frac{M_w \times R^2}{2}$$

$$L = T - V$$

$$T = \frac{1}{2} \times M_{eff} \times \dot{x}^2 + M_b \times l_b \times \dot{\theta} \times \cos\theta + \frac{1}{2} \times J_{eff} \times \dot{\theta}^2$$

$$M_{eff} = M_b + 2M_w + \frac{2J_w}{R^2}$$

$$J_{eff} = M_b \times l_b^2 + J_b$$

To run the state space algorithm, the bldc need to be linearised to torque vs voltage. Once the linearise has been done, MCU can simply use the formular below to get the voltage to fix the error,

$$u = -kx$$

where u means output voltage, k is the state constant for the robot to fix the error (x).

$$\text{Back-EMF Constant } (K_e) = kv \times \left(\frac{60}{2\pi}\right) = 220 \times \left(\frac{60}{2\pi}\right) = 0.04334V \cdot s/rad$$

$$\text{Torque Constant } (K_t) = K_e = 0.0434 \text{ N} \cdot \text{m/A}$$

$$\text{Target Torque } (T) = 0.3\text{kg/cm} = 0.0294 \text{ N} \cdot \text{m}$$

Phase Resistance: 2.55Ω (datasheet); measured as 2.9Ω , all the value here will follow according to the actual robot, so 2.9Ω will be used in calculation.

$$\text{Forces per Volt } (C_m) = \frac{2 \times K_t}{R_{ph} \times R_w}$$

$$\text{Back - EMF Viscous Damping } (C_v) = \frac{2 \times K_t^2}{R_{ph} \times R_w^2}$$

With the parameter included,

$$\text{Torque per Volt } (C_m) = \frac{2 \times 0.04334}{2.9 \times 0.026} = 1.15$$

$$\text{Back - EMF Viscous Damping } (C_v) = \frac{2 \times 0.04334^2}{2.9 \times 0.026^2} = 1.92$$

State space can be defined as $\dot{x} = A(X) + B(U)$

$$\text{Where the state vector } (X) = \begin{bmatrix} X \\ \dot{X} \\ \theta \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \text{Position} \\ \text{Linear Velocity} \\ \text{Pitch Radian} \\ \text{Pitch Velocity} \end{bmatrix} \text{ And U represent}$$

Voltage for the motor

$$U=-KX$$

$$U=\text{Voltage}$$

$$C1 = (M_b^2) \times g \times \frac{l_b^2}{d}$$

$$C2 = \frac{J_{eff}}{D \times R}$$

$$C3 = M_{eff} \times M_b \times g \times \frac{l_b}{D}$$

$$C4 = M_b \times \frac{l_b}{D \times R}$$

$$= \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & -(C2 * yes) & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$= \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

As $\sin \theta_b \approx 0$, and $\cos \theta_b \approx 1$, thus the equation can be simplified to

State vector (Q)= $\begin{bmatrix} Q \text{ Position} \\ Q \text{ Linear Velocity} \\ Q \text{ Pitch Radian} \\ Q \text{ Pitch Velocity} \end{bmatrix}$ will be tuning parameters for the K value; a

higher Q value means more importance on that vector.

Another tuning value will be R; it will determine the reaction for the system. A higher value will tell the system that the voltage is expensive, use less to save energy, a lower value will make the system react more strongly but use more energy to stay stable.

Parameter		Unit
Body Mass (M_b)	0.504	KG
Wheel Mass (M_w)=	0.0145+0.020	KG
Distance from wheel axis to the centre of mass(l_b)	Calculated in section below	cm
Radius of the wheel(r)	0.026	m
Gravity (g)	9.81	Kg.cm ²

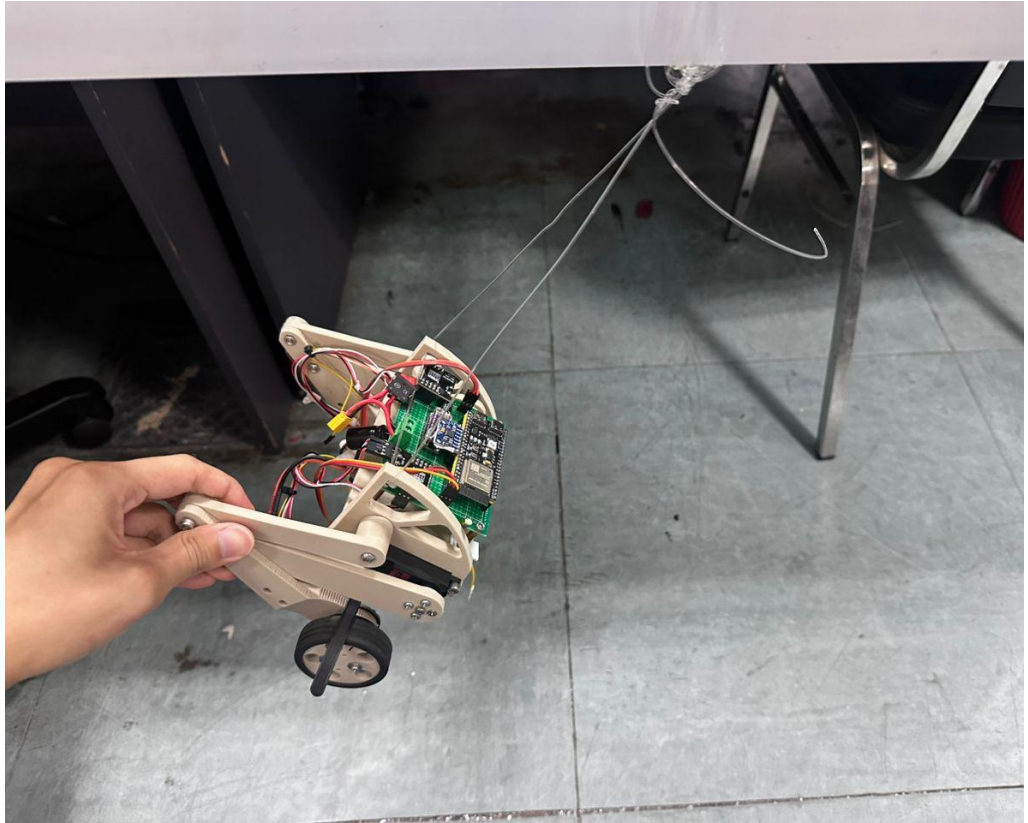
To calculate the movement of inertia (J_b), the pendulum period will be needed to calculate it. Pendulum period can be got through a simple experiment; the robot is hanging on a bar with the distance from the pivot point to Cog will be 30cm which 0.3m show as the figure below.



Next, the robot will be taken to 15° position and release show on the figure above, Start the timer when the robot starts swing. As the robot swings back and forth, this counts as one period, repeating this 10 times. Lastly, check the timer for 10 swings.

$$T = \frac{\text{total time for 10 swings}}{10}$$

Repeat 3 times for more accurate results. Pendulum period of the robot from the experiment is 1.15 seconds.



Once the pendulum period is calculated, Movement of inertia (J_b) can be calculated by the equation below as T =pendulum period.

$$J_b = M_b \times d \left(\frac{g \times T^2}{4\pi^2} - d \right)$$

$$J_b = 0.504 \times 0.3 \left(\frac{9.81 \times 1.15^2}{4\pi^2} - 0.3 \right)$$

$$J_b = 0.004334$$

To evaluate if the J_b value is correct, another equation can be used to check,

$$T = 2\pi \times \sqrt{\frac{I_{pivot}}{M_b \times g \times d}}$$

where $I_{pivot} = J_b + (M_b \times d^2)$, the equation will be come

$$T = 2\pi \times \sqrt{\frac{J_b + (M_b \times d^2)}{M_b \times g \times d}}$$

Put those parameters into the equation and get,

$$T = 2\pi \times \sqrt{\frac{0.004334 + (0.504 \times 0.3^2)}{0.504 \times 9.81 \times 0.3}}$$

$$T=1.15s$$

As the Movement of inertia (J_b) matches with the pendulum period, this means the Movement of inertia (J_b) getting from the equation was correct and able to put into the equation to calculate.

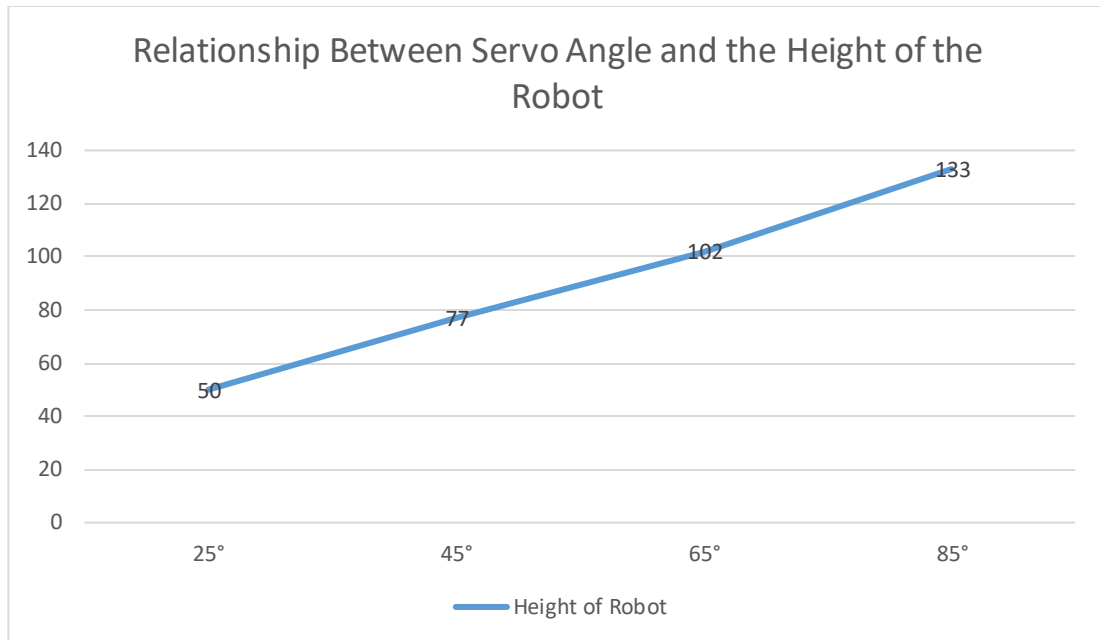
State space can be defined as

State vector (Q)= will be tuning parameters for the K value; a higher Q value means more importance on that vector.

Another tuning value will be R; it will determine the reaction for the system. A higher value will tell the system that the voltage is expensive, use less to save energy, a lower value will make the system react more strongly but use more energy to stay stable.

4.2 Variable geometry mathematic model build

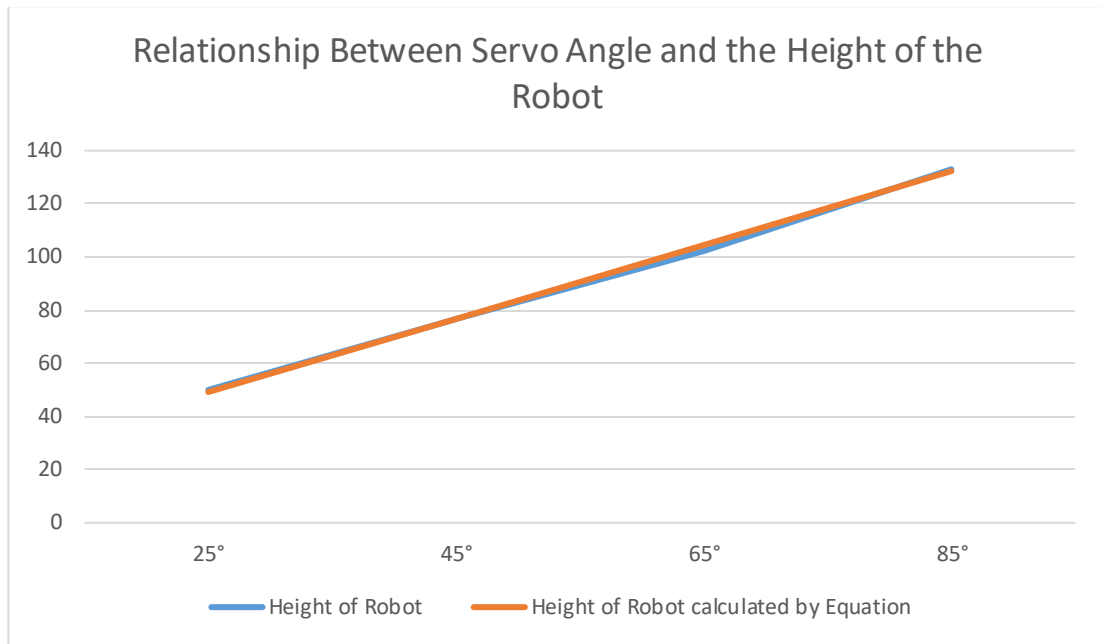
The robot has a hip motor to raise or lower the body up and down. in order to linearise the motion. The relationship between servo angle and the height of the robot will need derive into an equation.



The figure above shows that the relationship between servo angle and the height of the robot. The graph is almost linear in this case. And the equation of the relationship between servo angle and the height of the robot will be shown as below,

$$l_b = (1.385 * \text{Servo Angle}) + 14.575$$

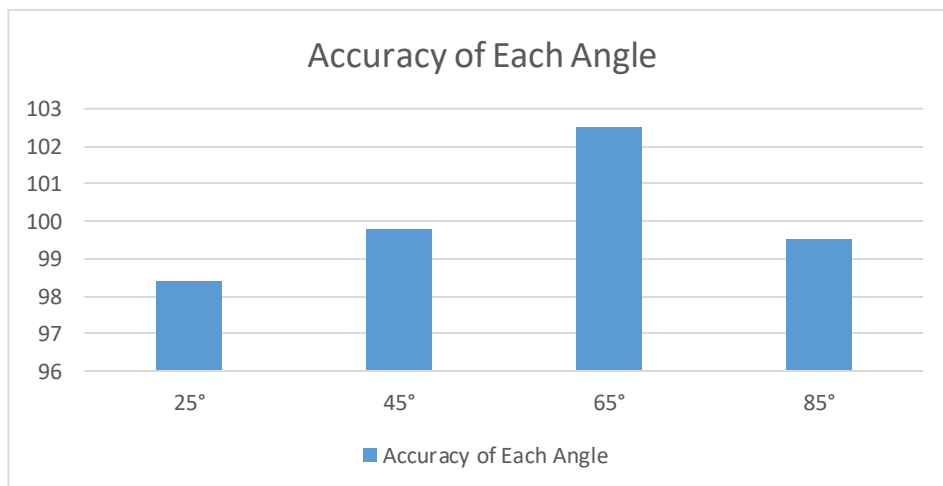
To evaluate the equation, the equation will be used to calculate the l_b and compare with the result above.



The figure above shows the height of the and height of the robot calculated by equation. The line is almost identical so the equation is highly accurate. The accuracy of each angle can be calculated by

$$Accuracy = \frac{\text{Height of Robot Calculated by Equation}}{\text{Height of Robot}} \times 100$$

The accuracy of each Servo Angle will show on the figure below.



How to convert servo angle to variable geometry

4.3 Free RTOS

An RTOS turns each program function into a task that is executed on a fixed routine. The reason an RTOS is used in this project is that it allows multiple processes to run at the same time, for example, the SimpleFOC motor driving runs at 1kHz to produce smooth motion for the 2804 gimbal BLDC motor, while the balance calculation runs at 200Hz so the I2C cycle can complete to read the sensor data and compute the voltage applied to the wheel motors. RTOS can also prioritize more important tasks, such as the balance calculation loop, over less important tasks, such as the servo movement task, which simply lifts the body of the robot.

4.4 Kalman Pitch estimation-not

<https://zh.wikipedia.org/zh-cn/%E5%8D%A1%E5%B0%94%E6%9B%BC%E6%BB%A4%E6%B3%A2>

To get a stable and clean pitch angle in radians for the robot to perform self-balancing, a low cost 6-axis motion tracking sensor, MPU6050 from InvenSense will be used in this project. The MPU6050 integrates 3-axis gyroscope and 3-axis accelerometer together. Gyroscope measures angular velocity (rate of rotation) and the accelerometer measures linear acceleration including gravity. Both sensors can provide pitch angles by deriving them from their raw data, but both sensors will introduce different kind of error.

The pitch angle derived from 3-axis gyroscope by integrating the angular rate over time, this is good to track rapid tilt action due to the gyroscope reacting instantly to the rotation instead of the linear acceleration. It appears that the gyroscope alone is sufficient, but it will introduce small bias which accumulates each time it is integrated because the gyroscope measures rate, not absolute angle.

Pitch angle can also be derived by the data of 3-axis accelerometer on MPU6050. When the robot is in stationary condition, the only force acting on the

sensor is gravity, which is constantly 9.81m/s^2 . The pitch angle will not drift like the pitch angle derived from gyroscope. This is because the accelerometer measures gravity directly at every sample. However, the robot will not always stay in stationary condition, it will need to move around and pitch back and forth to achieve the task. The accelerometer will catch that noise coming from the motor and robot base vibration. Although the data are only short-term error, the data become noisy and cause the robot to react to the noise instead of actual tilt.

Both sensors have their weaknesses. The gyroscope is good at high frequency but introduces drift at low frequency and the accelerometer is highly accurate on low frequency but becomes noisy when high frequency. A Kalman filter can be used in this scenario to fuse both signals and estimate an accurate pitch angle which keeps the strengths of both. In each 5 ms cycle, the Kalman filter uses the gyroscope rate to step the previous pitch estimate forward in time, then uses the accelerometer measurement to correct any error in that prediction. Then the corrected estimate will become next starting point for next cycle.

4.5 LQR Implementation-not

4.6 Four profile gain switches-not

4.7 Yaw loop-not

Yaw control are the loop out of the 4 state-space algorithm.it

4.8 Servo library

This is the library to drive the servo in the ESP32 environment. The reason for the custom library is ESP32 does not

The library uses the LEDC function of the ESP32, which is the ESP32's PWM generation peripheral; the PWM channel, resolution, and frequency can easily be set to match the specifications in the datasheet of the servo used in this project, the MG996R Servo Motor. The Servo Motor requires a 50Hz signal, which means 20ms per cycle. The servo motor sweeps from 0° to 180° with minimum and maximum pulse widths of 1000μs to 2000μs.

The full code will be placed in Appendix A; The MyServo.h library contains 4 functions that can be called and used in the main code. Before these functions can be called in the main code, the library needs to be included and instantiated. The library is instantiated with the command `MyServo Servo`. Here, `Servo` is a variable name that can be changed to differentiate between servo motors. For example, the left and right hip servo motors are named to *LeftServo* and *RightServo* in the main code

The first function will be the setup function. This function is called to initiate the servo motor under the Void Setup part. The function requires 3 parameters: the pin where the servo motor signal connects, and the minimum and maximum pulse widths in microseconds for the servo motor to sweep from 0° to 180°. Once the function receives those parameters, `ledcAttach` is used to set a 50Hz PWM signal output on the pin with 12-bit PWM resolution, and the minimum and maximum pulse widths are used in another function to set the angle of the servo motor.

The Write function will be the second function of the MyServo library. This function receives the angle as a parameter and converts it to the actual duty cycle of the PWM waveform needed to drive the servo motor to that position. The function maps 0° to 180° onto the pulse width range defined during the setup function using the `map` function. The mapping is shown below, with the pulse width denoted as Pulse Width in the equation

$$Pulse\ Width = Min\ Pulse + (Max\ Pulse - Min\ Pulse) \times \frac{Angle}{180}$$

For the ESP32-S3 to drive the servo motor, the duty cycle is the required parameter instead of the pulse width. The equation used to convert the pulse width to a duty cycle is shown below.

$$Duty\ cycle = \frac{pulse\ width * 4096 - 1}{20000}$$

As an example, to drive the MG996R used in this project to 90°, Equation 1 is first used to map the angle to a pulse width.

$$Pulse\ Width = 1000 + (2000 - 1000) \times \frac{90}{180}$$

$$Pulse\ Width = 1500\mu s$$

The conversion to a duty cycle is shown below.

$$\begin{aligned} Duty\ cycle &= \frac{1500 * (4096 - 1)}{20000} \\ &= 307.125 \end{aligned}$$

To verify the conversion, the inverse equation is shown below.

$$Pulse = \frac{Duty\ Cycle * 20000}{4095}$$

Substituting the duty cycle value for 90° into the equation gives:

$$Pulse = \frac{307.125 * 20000}{4095} = 1500\mu s$$

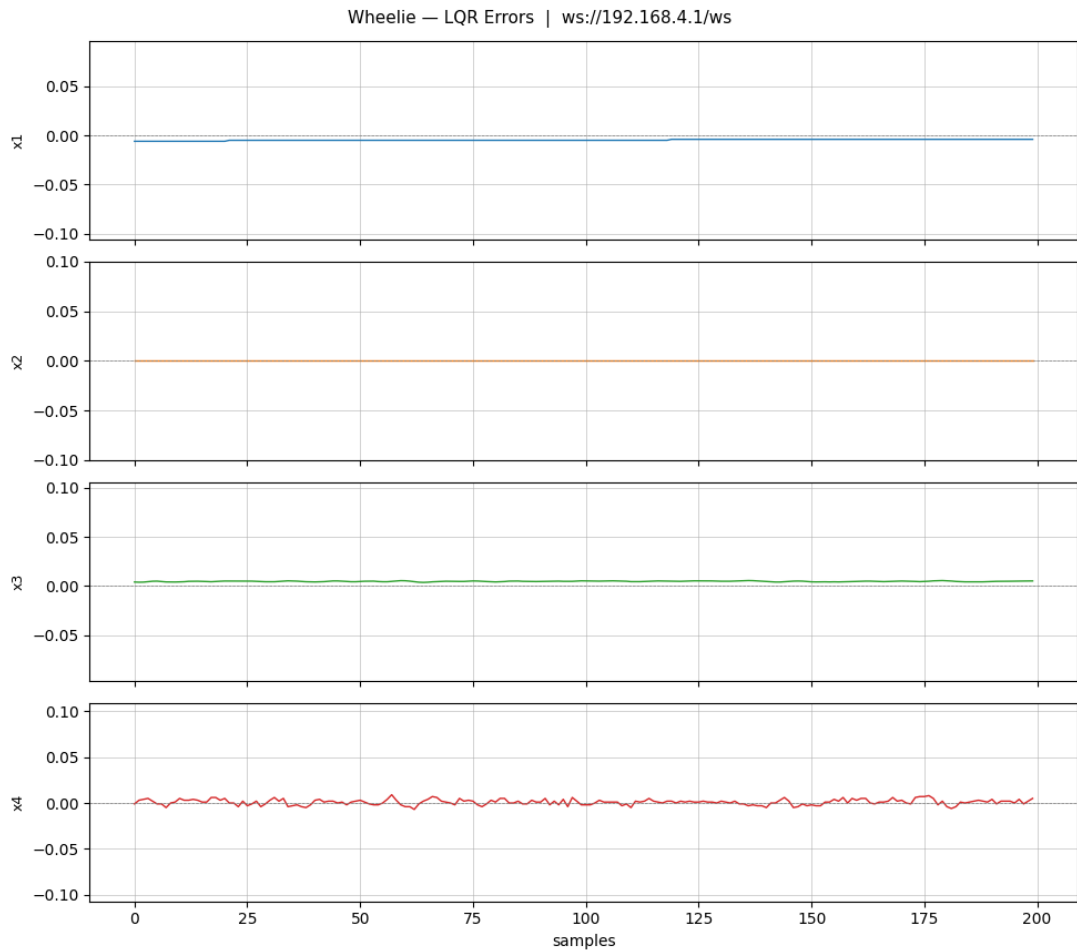
The speedControl function implements a slew-rate limiter on the servo command angle. Given a target angle and a speed in degrees per second, the function advances

the internal angle state by at most $\text{speed} \times \text{dt}$ degrees per invocation, where dt is the 100 ms servo task period. A 0.5° deadband prevents limit-cycling near the setpoint, and the step size is clamped to the remaining error to eliminate overshoot. The function is non-blocked by design — it executes a single incremental step and returns immediately, relying on the FreeRTOS servo task scheduler as its iteration mechanism rather than an internal delay () call. At a commanded speed of $60^\circ/\text{s}$, the maximum angular advance per task cycle is 6.0° , meaning a full-range transition from the squat profile to the stand profile spanning 60° completes in approximately one second — gradual enough for the LQR balance controller to track the shifting center of mass without saturating the motor voltage output.

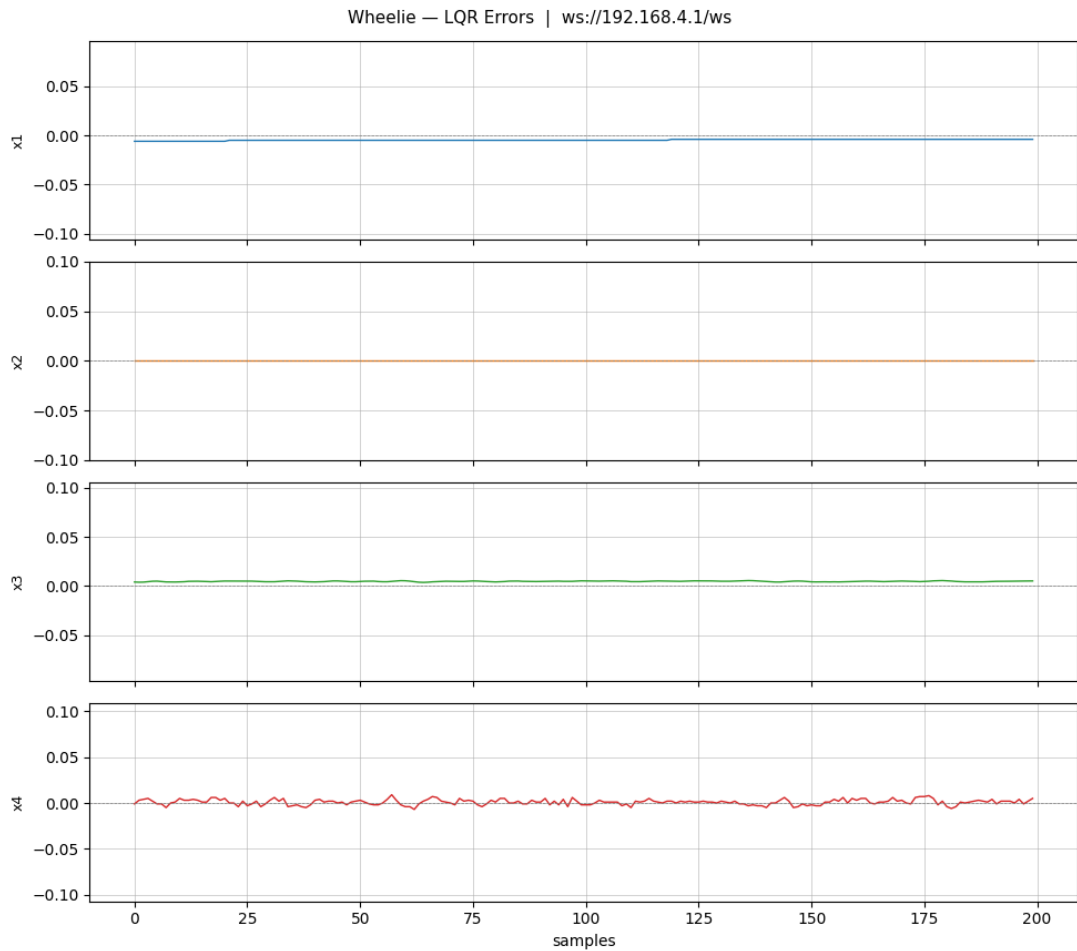
Lastly, there is the debug function of the MyServo library. This function prints the current angle and the current pulse width to the user via serial. It is useful for checking whether the pulse width setting matches the angle of the servo motor when the servo is not moving as expected.

CHAPTER 5 DATA PRESENTATION AND DISCUSSION OF FINDINGS

Robot self-balancing in different height profile

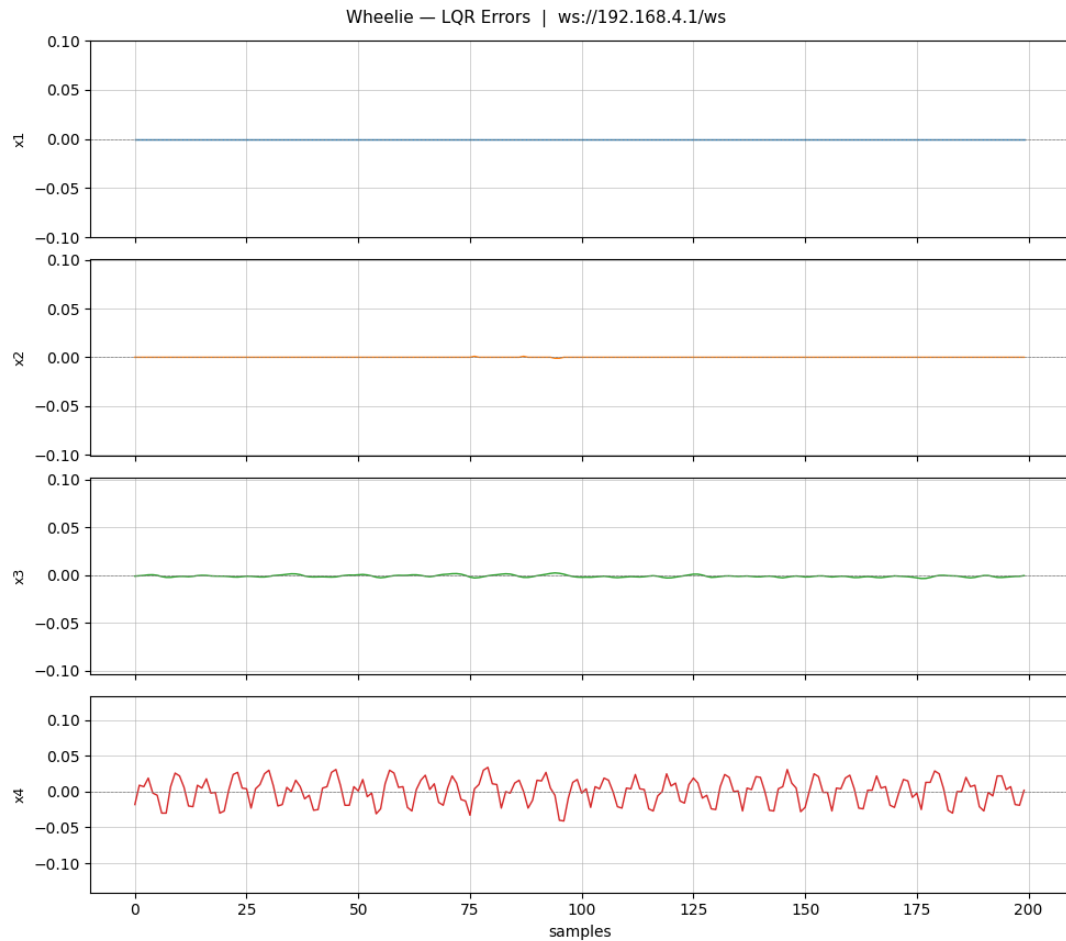


The figure above shows 4 state error of the robot between current state and the target state. x_1 indicates position error in meters, x_2 indicates velocity error in meter per seconds, x_3 indicates the error of the pitch angle in radian and the x_4 indicates the error of the radian per seconds. All the graphs are 200 sample windows and each sample windows will be 0.1 seconds. So, the graph will show 20 seconds of data. In this chapter this kind of graph will be used to indicate the robot movement



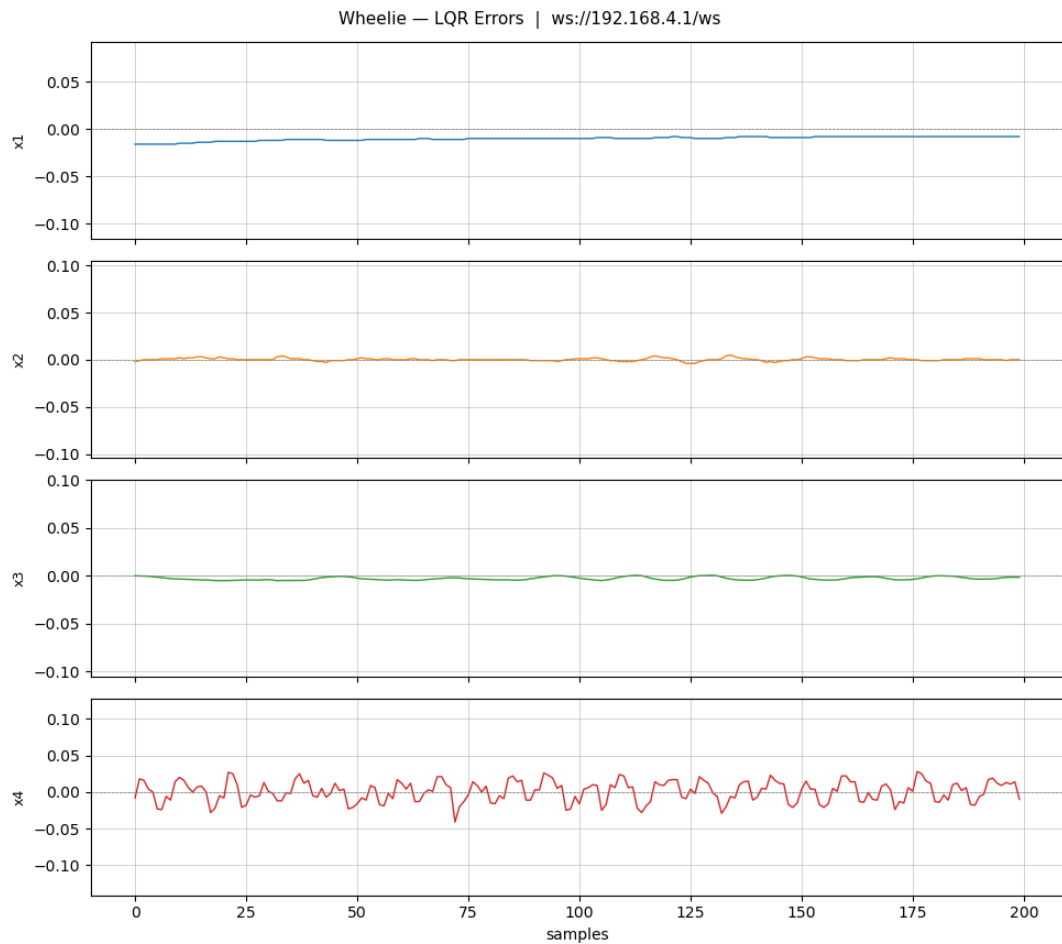
Robot self-balancing on squad down condition

As the graph above shows, the robot is very stable, and the error of the position will not more than 0.01m. and the velocity error (x_2) are 0 in this graph. Pitch angle errors are kept under 0.01 rad which are 0.57° in degree. As the last graph in the figure, it indicates the pitch rate. it is fluctuating around -0.02 to 0.02 rad/s are due to the robot are not a stable system as it needs to keep fixing to stay stable.



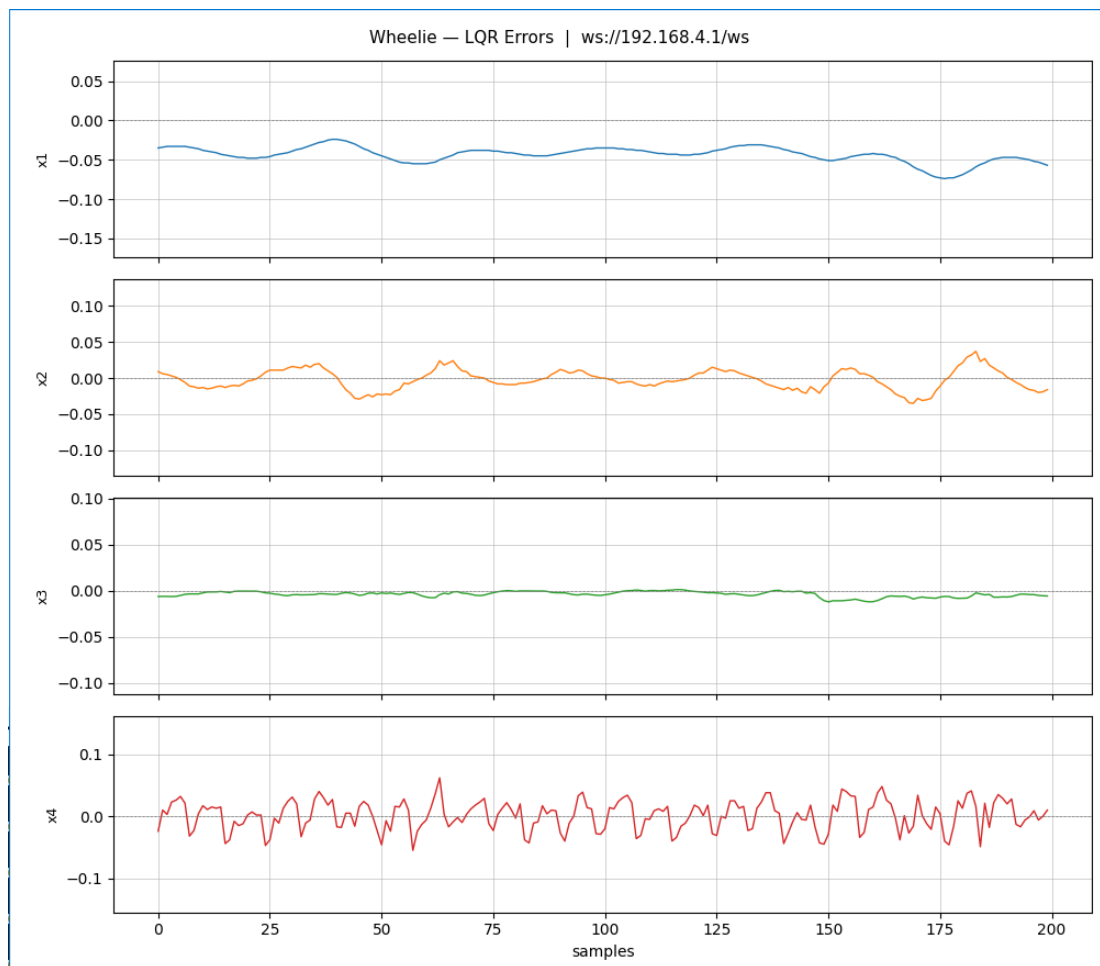
Robot self-balancing in stand-up condition (45 degree)

The figure above shows the error of the robot self-balancing in first stage stand up condition. The robot will be higher than the squat down condition as this will cause the system to become more unstable. The unstable can be easily noticed on the pitch rate error (x_4). It grows to -0.4 to 0.4 rad/s which is double of the fluctuation on squat down condition. Other than that, x_1 to x_3 error didn't change much compared to the squat down condition.



Robot self-balancing in stand up condition (65 degree)

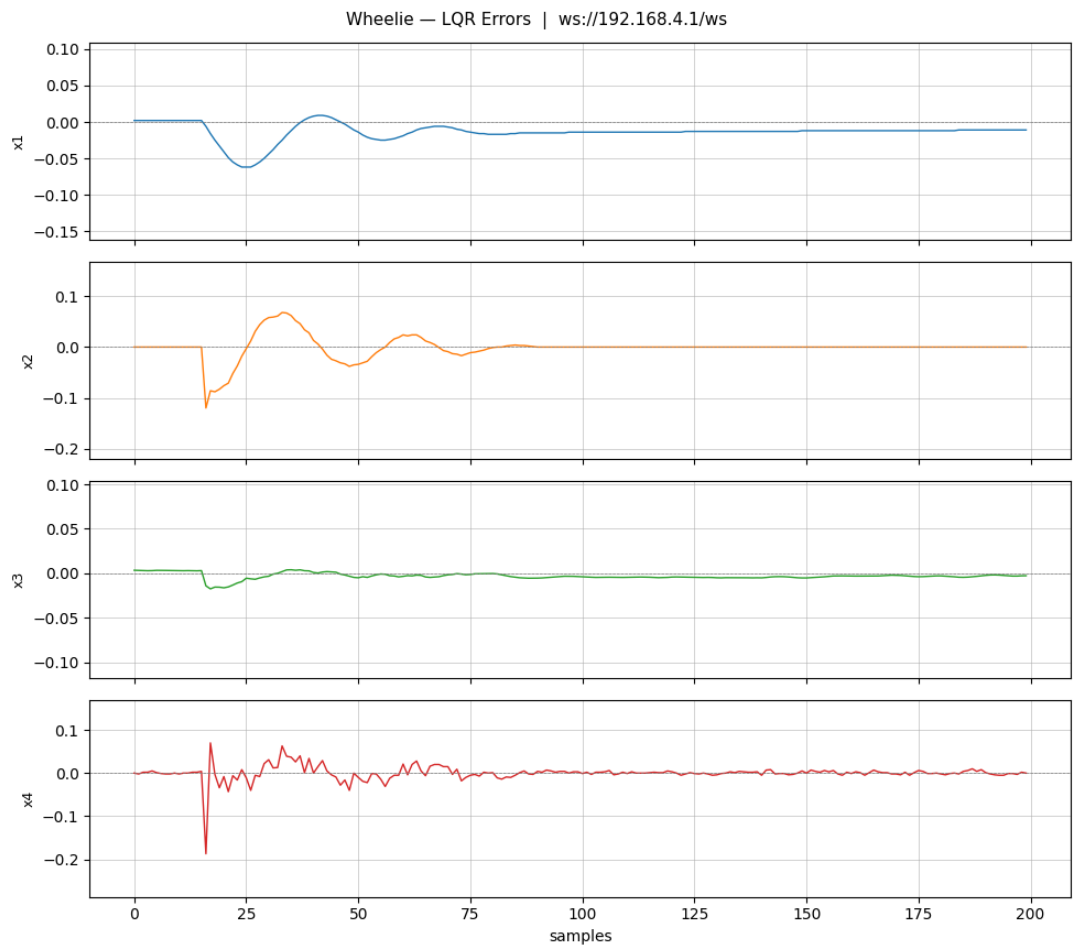
The figure above shows the same 4 state error for the robot with 104.5mm between CoG and the wheel axis. The position error (x_1) were fluctuating more than the squat down condition. And the velocity error (x_2) are



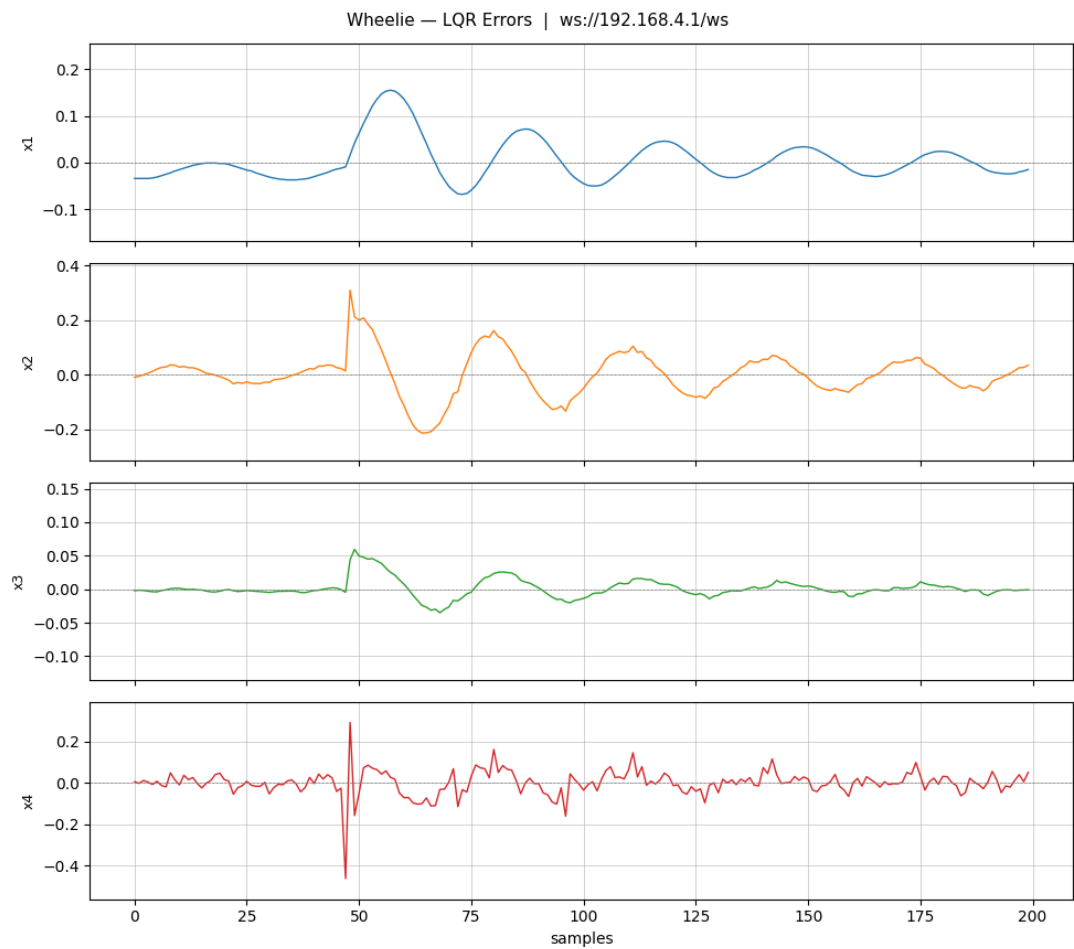
Robot self-balancing in stand-up condition (85 degree)

The figure above shows the 4-state error of the robot in fully stand-up condition which has a furthest distance between CoG and wheel axis, 132.3mm. Position error (x_1) are oscillating between -0.07m and -0.02m compared to squat down conditions without oscillating. The wave on position error (x_1) also show that the second drift integrator are actively against to the position error and cause the robot to oscillate back and forth gradually. The oscillation for the velocity error (x_2) is noticeable more active compare to other state, reach $\pm 0.03\text{m/s}$. this show the wheel motor are working with more effort to stay stable for the robot at this height. The pitch angle

The robot can still stay stable but the position error compared to squat down condition will be bigger as the Cog are higher and cause unstable.



Robot repone to lightly tap when self-balancing in squad down condition.



Robot repone to lightly tap when self-balancing in stand-up condition.

The robot will have some oscillating when being tap by moderate force as the 2804 gimbal motor cannot generate so much force to pull the robot back. So, the robot will move back and forth to dilute the force and back to stable condition slowly.

Robot accelerate in linear velocity

Robot rotating in a linear velocity

CHAPTER 6 CONCLUSIONS

6.1 Summary and Conclusions-not

6.2 Areas of Future Research -not

Swap to the 3528 motor to increase the torque to hold the robot on different angle, as current motor cannot set the pitch rad being too aggressive due to current 2804 gimbal motor are slightly lower on torque.

Change to a magnetic encoder servo motor, current servo motor has getting difference angle when giving same angle command

Model predictions control for robots to actively change with the load. More complexity but can easilt adopt to handle non linear system.

REFERENCES-not

1. Grasser, F., D'Arrigo, A., Colombi, S., & Rufer, A. (2002). JOE: a mobile, inverted pendulum. *IEEE Transactions on Industrial Electronics*, 49(1), 107–114. <https://doi.org/10.1109/41.982254>
2. Harly, M., Marji, Yoto, & Paryono. (2021). Model of integrated of car Brake-Active suspension of quarter car model by using LQR control under periodic road excitation. *IOP Conference Series Materials Science and Engineering*, 1152(1), 012023. <https://doi.org/10.1088/1757-899x/1152/1/012023>
3. Jiménez, A., Jiménez, F., & Ruge, I. (2020). Modeling and Control of a Two Wheeled Self-Balancing Robot: a didactic platform for control engineering education. *Proceedings of the 18th LACCEI International Multi-Conference for Engineering, Education, and Technology: Engineering, Integration, and Alliances for a Sustainable Development* “Hemispheric Cooperation for Competitiveness and Prosperity on a Knowledge-Based Economy.” <https://doi.org/10.18687/laccei2020.1.1.556>
4. Monroy-Bautista, L. N., Rivera-Ruiz, J. R., Cruz-Miguel, E. E., Pérez-Juárez, J. G., García-Martínez, J. R., & García-Arellano, C. (2025). Balancing robot control via State-Space approach with evolutionary dynamic parameter identification (pp. 1–6). <https://doi.org/10.1109/enc68268.2025.11311927>
- 5.

APPENDIX A

Custom Servo Library using LEDC function on ESP32 environment

MyServo.h

```
#ifndef MY_SERVO_H
```

```
#define MY_SERVO_H
```

```
#include <Arduino.h>
```

```
class MyServo {
```

```
private:
```

```
    int _pin;
```

```
    int _min_us;
```

```
    int _max_us;
```

```
    float _current_angle;
```

```
    const char* _name; // Add a name to identify which servo is talking
```

```
public:
```

```
    MyServo(const char* name); // Updated constructor
```

```
    void setup(int pin, int min_us, int max_us);
```

```
    void write(int angle);
```

```
void speedControl(int target_angle, float speed);
```

```
// The Debug Function
```

```
void debug();
```

```
};
```

```
#endif
```

MyServo.cpp

```
#include "MyServo.h"
```

```
MyServo::MyServo(const char* name) {
```

```
    _name = name;
```

```
    _current_angle = 0;
```

```
}
```

```
void MyServo::setup(int pin, int min_us, int max_us) {
```

```
    _pin = pin;
```

```
    _min_us = min_us;
```

```
    _max_us = max_us;
```



```

    ledcAttach(_pin, 50, 12);

}

void MyServo::write(int angle) {

    _current_angle = angle;

    int us = map(angle, 0, 180, _min_us, _max_us);

    int pulse = (us * 4095) / 20000;

    ledcWrite(_pin, pulse);

}

void MyServo::speedControl(int target_angle, float speed) {

    while(abs(_current_angle - target_angle) > 0.5) {

        if(_current_angle < target_angle) _current_angle += speed;

        else _current_angle -= speed;

        int us = map((int)_current_angle, 0, 180, _min_us, _max_us);

        int pulse = (us * 4095) / 20000;

        ledcWrite(_pin, pulse);

        delay(20);

    }

    write(target_angle);

```

```
}

void MyServo::debug() {

    int us = map((int)_current_angle, 0, 180, _min_us, _max_us);

    printf("[%s] Angle: %.2f | Pulse: %d us\n", _name, _current_angle, us);

}
```